DEEP LEARNING

Function Approximation MLP

2018

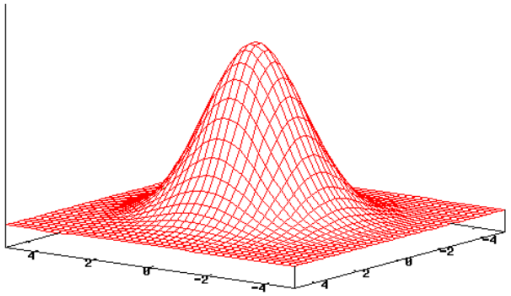
Contents

Contents 2
1. Data Preparation 2
1.1. Results 3
1.2. Generate and Save Dataset 3
1.3. Load and Draw 3
Support Functions 4
2. Multi-Layer Perceptron Neural Networks 5
2.1. General MLP Model with N Layers and H_i Hidden Nodes 5
2.2. Building Model 5
Support Functions 5
2.3. Training and Predicting Process 6
Support Functions 7
3. Experiments 9
3.1. Experiment 1: Performance against the # training samples 9
Support Functions 9
3.2. Experiment 2: Performance against the # hidden nodes 10
Support Functions 10
3.3. Experiment 3: Performance against the # hidden layers 11
Support Functions 12
References 12

1. Data Preparation

- Given a 2-D Gaussian Function

$$f(x) = \frac{1}{(2\pi)^{1/2}[det(C_X)]^{1/2}} exp\left(-\frac{1}{2}(x-\mu_X)^T C_X^{-1}(x-\mu_X)\right)$$

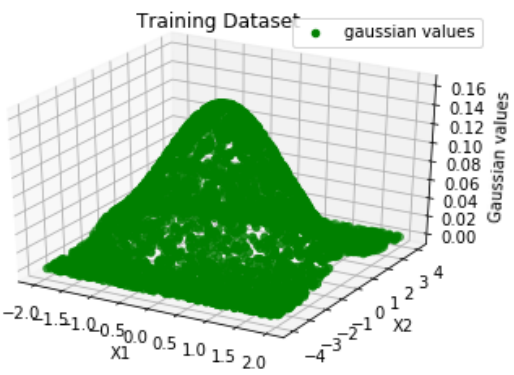


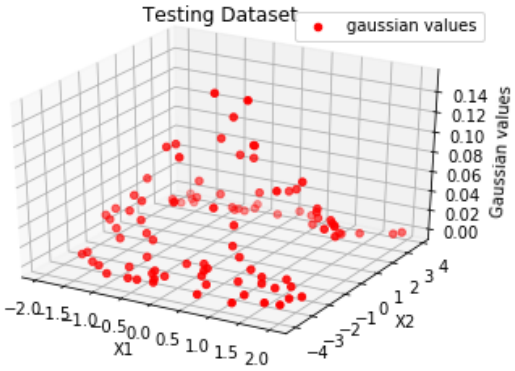
- Generate A training samples $X_i = (x_{i_1}, x_2, y_i)$ randomly

$-2 \leq x_{i_1} \leq 2, -4 \leq x_{i_2} \leq 4, 0 \leq y_i \leq 1$

- Generate 100 test samples similarly

1.1. Results





training-data.csv

```
x1,y1,y
-0.384539080085,1.4010284453,0.0553957407069
-1.77619053269,-1.58828471197,0.00931035894406
0.715266942509,1.56344790858,0.0363026278651
-0.519593136699,3.40457175732,0.000422880888674
1.19565868823,2.37884302654,0.00459804535669
-0.787584371878,-2.11430006074,0.0124859157334
-0.493555624268,-1.42907780883,0.0507518435616
-0.842318808795,3.13904047789,0.000809239924905
0.0164010869208,3.23115511459,0.000860331092525
```

testing-data.csv

```
x1,y1,y
-1.48193363893,0.165306451342,0.0523604453242
-1.06398814389,1.66860774068,0.0224595728589
0.588205614113,-1.79836517141,0.0265711660685
1.25600275585,2.34338512574,0.00464308876764
1.00631489298,-0.691277617661,0.07553612422
0.414914538163,3.63749939964,0.000195557707204
0.127595192953,1.00290898887,0.095471209035
1.20246095971,-2.20991762148,0.00671977106272
-0.692572818063,2.93441153792,0.00168989115491
0.346640096184,-2.44148030908,0.00760940882292
```

1.2. Generate and Save Dataset

```
n_training_samples = 5000
n_testing_samples = 100
rangex1 = [-2,2]
rangex2 = [-4,4]
mean    = [0,0]
cov     = [[1,0],[0,1]]

(x_train, y_train) = generate_dataset(samples = n_training_samples, dim = 2, rangex1 = rangex1, rangex2 =
rangex2, mean=mean, cov=cov)
save_dataset(x_train, y_train, filename='training-data.csv')
(x_test, y_test) = generate_dataset(samples = n_testing_samples, dim = 2, rangex1 = rangex1, rangex2 = rangex2,
mean=mean, cov=cov)
save_dataset(x_test, y_test, filename='testing-data.csv')
```

1.3. Load and Draw

```
# IMPORT LIBRARIES
import numpy as np
from scipy.stats import multivariate_normal

from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
from matplotlib import cm
import csv, time

from tqdm import tqdm
from IPython.display import clear_output
```

```
np.set_printoptions(precision=2, suppress=True, formatter={'float': '{: f}'.format}, linewidth=1000)
```

Support Functions

```
def multivariate_gaussian(x, mean, cov):
    # np.multiply
    # --> (x-mean)^T .* cov^-1 .* (x-mean): x --> one value
    # --> (((x-mean) .* cov^-1) * (x-mean)) sum (axis=1): x --> multiple value
    return (1.0/(((2*np.pi)**(len(mean)/2.0))*(np.linalg.det(cov)**0.5)))*np.exp(-0.5 * np.multiply(np.matmul((x
- mean),np.linalg.inv(cov)),(x - mean)).sum(axis=1))
# multivariate_gaussian

def generate_dataset(samples = 100, dim = 2, rangex = [-3,3], rangey = [0,1], mean=[0,0], cov=[[1,0],[0,1]]):
    x = np.random.rand(samples, dim)*(rangex[1] - rangex[0]) + rangex[0]
    y = multivariate_gaussian(x, mean,cov)
    return (x, y)
# generate_dataset

def save_dataset(x, y, filename='training-data.csv', headers = ['x1','y1','y'], separator=','):
    save_file = open(filename, 'wt')
    csv_file = csv.writer(save_file, delimiter=separator, quotechar='|', quoting=csv.QUOTE_MINIMAL,
lineterminator='\n')
    csv_file.writerow(headers)
    for i in range(len(x)):
        csv_file.writerow([x[i, 0], x[i, 1], y[i]])
    save_file.close()
# save_dataset

def load_dataset(filename='training-data.csv', headers = ['x1','y1','y'], separator=','):
    read_file = open(filename, 'rt')
    csv_file = csv.reader(read_file, delimiter=separator, quotechar='|', quoting=csv.QUOTE_MINIMAL,
lineterminator='\n')
    x = []
    y = []
    next(csv_file)
    for row in csv_file:
        x.append([row[0], row[1]])
        y.append(row[2])
    read_file.close()
    return (np.array(x).astype(dtype=np.double), np.array(y).astype(dtype=np.double))
# save_dataset

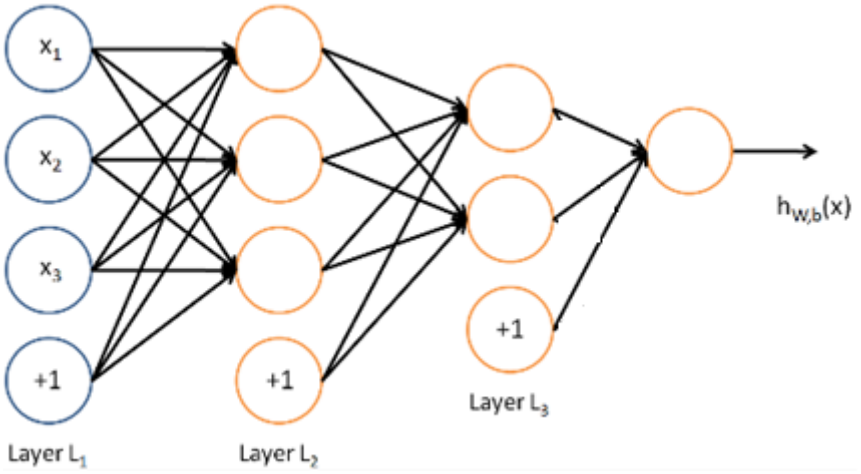

def plot_2d_dataset(x, y, title = 'Training', label='gaussian values', color = 'green', save_image = ''):
    fig = plt.figure()
    ax  = fig.gca(projection='3d')

    surf = ax.scatter(x[:,0], x[:,1], y, c = color, label = label)
    ax.set_xlabel('X1')
    ax.set_ylabel('X2')
    ax.set_zlabel('Gaussian values')
    plt.legend()
    plt.title(title)

    if save_image != '':
        fig.savefig(save_image)
    plt.show()
    pass
# plot_2d_dataset
```

2. Multi-Layer Perceptron Neural Networks

2.1. General MLP Model with N Layers and H_i Hidden Nodes



2.2. Building Model

```
import tensorflow as tf

# Parameters of MLP
n_input   = 2
n_classes = 1
n_layers  = 2
n_hiddens = [128, 128]

# Parameters of Training
learning_rate = 0.0001
batch_size = 32
epochs = 200

(x_input, y_output, mlp_net, f_loss, train_step) = init_multilayer_perceptron(n_input, n_classes, n_layers,
n_hiddens)
# Init session
sess = tf.Session()
# Initialize variables
init = tf.global_variables_initializer()
```

```
--------------------------------
MLP MODELS
--------------------------------
Input: Tensor("x_input_14:0", shape=(?, 2), dtype=float32)

Layer 0:
+ Weight: <tf.Variable 'Variable_84:0' shape=(2, 128) dtype=float32_ref>
+ Bias: <tf.Variable 'Variable_85:0' shape=(128,) dtype=float32_ref>
+ Layer: Tensor("Sigmoid_42:0", shape=(?, 128), dtype=float32)
Layer 1:
+ Weight: <tf.Variable 'Variable_86:0' shape=(128, 128) dtype=float32_ref>
+ Bias: <tf.Variable 'Variable_87:0' shape=(128,) dtype=float32_ref>
+ Layer: Tensor("Sigmoid_43:0", shape=(?, 128), dtype=float32)
Layer 2:
+ Weight: <tf.Variable 'Variable_88:0' shape=(128, 1) dtype=float32_ref>
+ Bias: <tf.Variable 'Variable_89:0' shape=(1,) dtype=float32_ref>
+ Layer: Tensor("Sigmoid_44:0", shape=(?, 1), dtype=float32)

Output: Tensor("Sigmoid_44:0", shape=(?, 1), dtype=float32)
```

Support Functions

```
def build_multilayer_perceptron(x, n_classes = 1, n_layers = 2, n_hiddens = [5, 10]):
    # Network parameters
    n_input  = int(x.shape[1])
    n_output = n_classes

    # weights definition
    weights = []
    biases  = []
```

```
    layers  = []

    # (None, n_input) x (n_input, n_hiddens[0])
    print("--------------------------------")
    print("MLP MODELS")
    print("--------------------------------")
    print('Input: %s\n'%(x))
    weight_0 = tf.Variable(tf.random_normal([n_input, n_hiddens[0]], dtype=tf.float32))
    bias_0   = tf.Variable(tf.random_normal([n_hiddens[0]], dtype=tf.float32))
    weights.append(weight_0)
    biases.append(bias_0)

    layer_0 = tf.add(tf.matmul(x, weights[0]), biases[0])
    layer_0 = tf.nn.sigmoid(layer_0)
    layers.append(layer_0)
    print('Layer %s:\n+ Weight: %s\n+ Bias: %s\n+ Layer: %s'%(0, weights[0], biases[0], layers[0]))

    for i in range(1, n_layers):
        weight_i = tf.Variable(tf.random_normal([n_hiddens[i-1], n_hiddens[i]], dtype=tf.float32))
        bias_i   = tf.Variable(tf.random_normal([n_hiddens[i]], dtype=tf.float32))
        weights.append(weight_i)
        biases.append(bias_i)

        layer = tf.add(tf.matmul(layers[i-1], weights[i]), biases[i])
        layer = tf.nn.sigmoid(layer)
        layers.append(layer)
        print('Layer %s:\n+ Weight: %s\n+ Bias: %s\n+ Layer: %s'%(i, weights[i], biases[i], layers[i]))
    # for

    weight_out = tf.Variable(tf.random_normal([n_hiddens[n_layers-1], n_output], dtype=tf.float32))
    bias_out = tf.Variable(tf.random_normal([n_output], dtype=tf.float32))
    weights.append(weight_out)
    biases.append(bias_out)

    layer_out = tf.add(tf.matmul(layers[n_layers-1], weights[n_layers]), biases[n_layers])
    layer_out = tf.nn.sigmoid(layer_out)
    layers.append(layer_out)
    print('Layer %s:\n+ Weight: %s\n+ Bias: %s\n+ Layer: %s'%(n_layers, weights[n_layers], biases[n_layers],
layers[n_layers]))
    print('\nOutput: %s'%(layer_out))


    # layer_out
    return layer_out
# def build_multilayer_perceptron

def init_multilayer_perceptron(n_input   = 2, n_classes = 1, n_layers  = 2, n_hiddens = [10, 4]):
    # Netowrk Architecture
    x_input = tf.placeholder(tf.float32, [None, n_input], name="x_input")
    y_output = tf.placeholder(tf.float32, [None, n_classes], name="y_output")

    mlp_net = build_multilayer_perceptron(x_input, n_classes = n_classes, n_layers = n_layers, n_hiddens =
n_hiddens)

    # Loss function
    f_loss = tf.reduce_sum(tf.pow(y_output - mlp_net, 2))

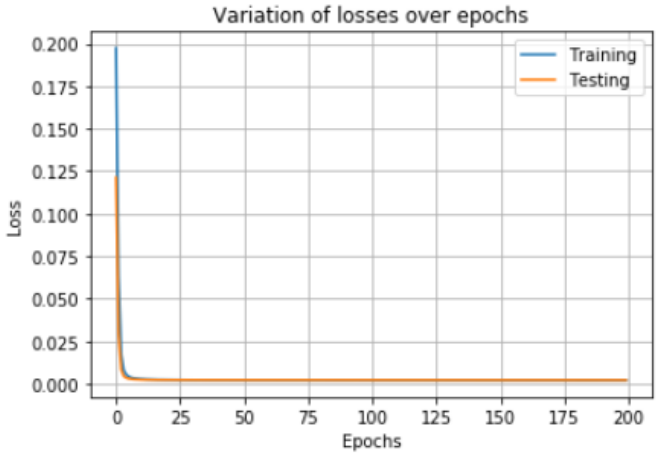
    # Algorithm MLP
    train_step = tf.train.GradientDescentOptimizer(learning_rate).minimize(f_loss)

    return (x_input, y_output, mlp_net, f_loss, train_step)
# def init_multilayer_perceptron
```

2.3. Training and Predicting Process

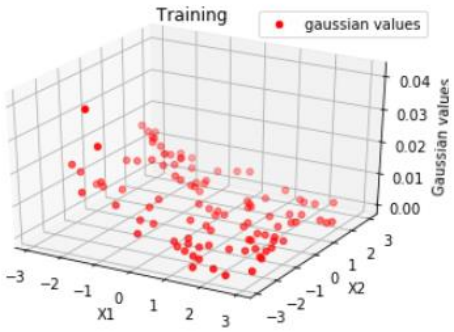
```
(train_loss, val_loss, loss_logs, val_loss_logs, _, _) = traininig_multilayer_perceptron(x_train, y_train,
x_test, y_test, x_input, y_output, f_loss, train_step, batch_size, epochs, learning_rate,
save_image='gauss_train.png')
```

```
Best valid loss at epoch 199 with val_loss 0.002120
```

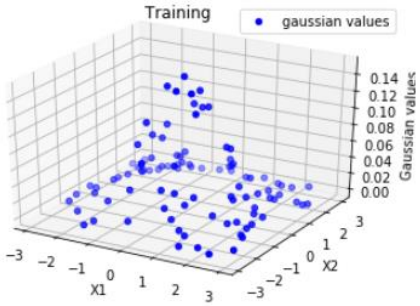



```
loss, y_pre = predict_multilayer_perceptron(x_test, y_test, mlp_net, f_loss)
print('Predict')
plot_2d_dataset(x_test, y_pre.reshape(-1), color = 'red',save_image='gauss_train_predict.png')
print('Test')
plot_2d_dataset(x_test, y_test, color = 'blue',save_image='gauss_train_test.png')
```

```
Predict
```




```
Test
```



Support Functions

```
def traininig_multilayer_perceptron(x_train, y_train, x_test, y_test, x_input, y_output, f_loss, train_step,
batch_size = 16, epochs = 2, learning_rate = 0.0001, log_file_name='gauss.log.csv',
save_model='./checkpoints/model.ckpt', sample_interval=10000, time_sleep = 1.0, save_image=''):
    # TRAINING PROCESS
    print('\nTRAINING MLP:')

    steps_per_epoch = int(len(x_train) / batch_size)
    total_training_samples = len(x_train)
    total_valid_samples = len(x_test)
    print('+Total training samples: %s'%(total_training_samples))
    print('+Total validating samples: %s'%(total_valid_samples))

    # Add ops to save and restore all the variables.
    # Initialize variables
    sess.run( tf.global_variables_initializer())
    # saver = tf.train.Saver()

    # Open log files
    log_file = open(log_file_name, 'wt')
    csv_log_file = csv.writer(log_file, delimiter=',', quotechar='|', quoting=csv.QUOTE_MINIMAL,
lineterminator='\n')
    csv_log_file.writerow(['epoch','loss','val_loss'])
    log_file.flush()

    loss_logs = [] # [epoch, loss]
    val_loss_logs = [] # [epoch, val_loss]

    best_val_loss = -1
    best_epoch = 0

    for epoch in range(epochs):
        clear_output(wait = True)
        pbar = tqdm(range(steps_per_epoch))
```

```
        pbar.set_description("Epoch %s/%s"%(epoch, epochs))
        for step in pbar:

            # Select a random batch of training set
            train_idx = np.random.randint(0, x_train.shape[0], batch_size)
            x_train_batch = x_train[train_idx]
            y_train_batch = y_train[train_idx].reshape(-1, 1)

            _, loss = sess.run([train_step, f_loss], feed_dict = {x_input: x_train_batch, y_output:
y_train_batch})
        # for step_per_epochs

        # Training
        train_loss= sess.run(f_loss, feed_dict = {x_input: x_train, y_output: y_train.reshape(-
1,1)})/total_training_samples

        # Validation
        val_loss = sess.run(f_loss, feed_dict = {x_input: x_test, y_output: y_test.reshape(-
1,1)})/total_valid_samples

        print("Train Loss: %s - Val loss: %s"%(train_loss, val_loss), end="")

        if (best_val_loss==-1) or (best_val_loss>val_loss):
            best_val_loss = val_loss
            best_epoch = epoch
            print(' with best %.5f'%(best_val_loss), end="")

        csv_log_file.writerow([epoch, train_loss, val_loss])
        log_file.flush()

        loss_logs.append([epoch, train_loss])
        val_loss_logs.append([epoch, val_loss])
        pbar.close()

        if (epoch+1) % sample_interval == 0:
            plot_loss(epoch, loss_logs, val_loss_logs)
            time.sleep(time_sleep)
    # for epoch

    clear_output(wait = True)
    # Save the variables to disk.
    # save_path = saver.save(sess, save_model, global_step=step)
    # print("\nModel saved in path: %s" % save_path)
    print('\nBest valid loss at epoch %s with val_loss %f'%(best_epoch, best_val_loss))

     # Training
    train_loss= sess.run(f_loss, feed_dict = {x_input: x_train, y_output: y_train.reshape(-
1,1)})/total_training_samples
    # Validation
    val_loss = sess.run(f_loss, feed_dict = {x_input: x_test, y_output: y_test.reshape(-
1,1)})/total_valid_samples
    plot_loss(loss_logs, val_loss_logs, save_image = save_image)
    print('Final train loss: %s - val loss: %s'%(train_loss, val_loss))
    log_file.close()

    return (train_loss, val_loss, loss_logs, val_loss_logs, best_epoch, best_val_loss)
# def traininig_multilayer_perceptron

def plot_loss(loss_logs, val_loss_logs, xlabel = 'Epochs', ylabel = 'Loss', title='Variation of losses over
epochs', save_image=''):
    # Convert the log lists to numpy arrays
    loss_logs     = np.array(loss_logs)
    val_loss_logs = np.array(val_loss_logs)

    fig = plt.figure()

    plt.plot(loss_logs[:,0], loss_logs[:,1], label="Training")
    plt.plot(val_loss_logs[:,0], val_loss_logs[:,1], label="Testing")
    plt.xlabel(xlabel)
    plt.ylabel(ylabel)
    plt.legend()
    plt.title(title)
    plt.grid(True)
    if save_image != '':
        fig.savefig(save_image)
    plt.show()
# plot_loss

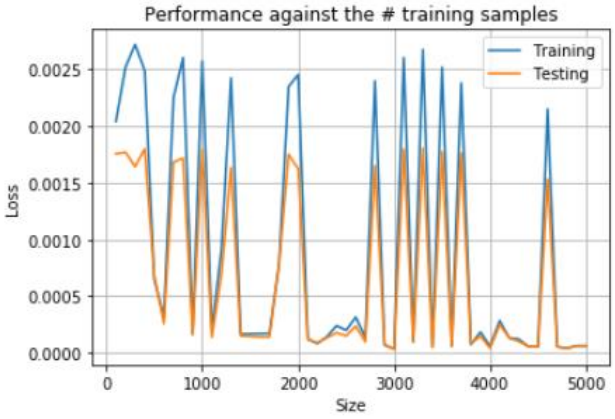
def predict_multilayer_perceptron(x_test, y_test, mlp_net, f_loss):
```

```
    y_predict = sess.run(mlp_net, feed_dict = {x_input: x_test})
    loss = sess.run(f_loss, feed_dict = {x_input: x_test, y_output: y_test.reshape(-1,1)})/len(x_test)
    return (loss, y_predict)
# predict_multilayer_perceptron
```

3. Experiments

3.1. Experiment 1: Performance against the # training samples

- Generate random from the original dataset (5000 samples) with the number of samples 100, 200, …5000
- MLP with 1 layer with 128 hidden nodes
- Training 100 epochs, 32 batch size



experiment1.csv

```
size,loss,val_loss
100,0.00204260200262,0.0017572183907
200,0.00252178996801,0.00176959142089
300,0.0027210851510 4,0.00164328843355
400,0.00249105259776,0.0018003834784
500,0.000648247301579,0.000680786222219
600,0.000315085798502,0.000258906632662
700,0.00225273098264,0.00168310075998
800,0.00260511815548,0.00171734809875
900,0.000165065725644,0.000163155719638
1000,0.00257457304001,0.001795347929
1100,0.000193892554803,0.000137382699177
1200,0.000914504130681,0.000694066882133
1300,0.0024253823207,0.00163449943066
1400,0.000166280386703,0.000149409938604
```

Increase size, loss is decrease. But it has a little bit for unstable training process for initialize random data.

Support Functions

```
# Parameters of MLP
n_input   = 2
n_classes = 1
n_layers  = 1
n_hiddens = [128, 128]

# Parameters of Training
learning_rate = 0.01
batch_size = 32
epochs = 100

step_size = 100
from_size = 100
final_size = 5000

def get_number_samples(x, y, samples = 100):
    # Select samples number of dataset set
    idx = np.random.randint(0, x.shape[0], samples)
    return (x[idx],y[idx])
# def get_number_samples

def experiment1(save_image='experiments1.png', log_file_name = 'experiment1.csv'):
    global n_input, n_classes, n_layers, n_hiddens
    global learning_rate, batch_size, epochs
    global step_size, from_size, final_size
    # sess.close()
    (x_input, y_output, mlp_net, f_loss, train_step) = init_multilayer_perceptron(n_input, n_classes, n_layers,
n_hiddens)

    # Open log files
    log_file = open(log_file_name, 'wt')
    csv_log_file = csv.writer(log_file, delimiter=',', quotechar='|', quoting=csv.QUOTE_MINIMAL,
lineterminator='\n')
    csv_log_file.writerow(['size','loss','val_loss'])
```

```
    log_file.flush()

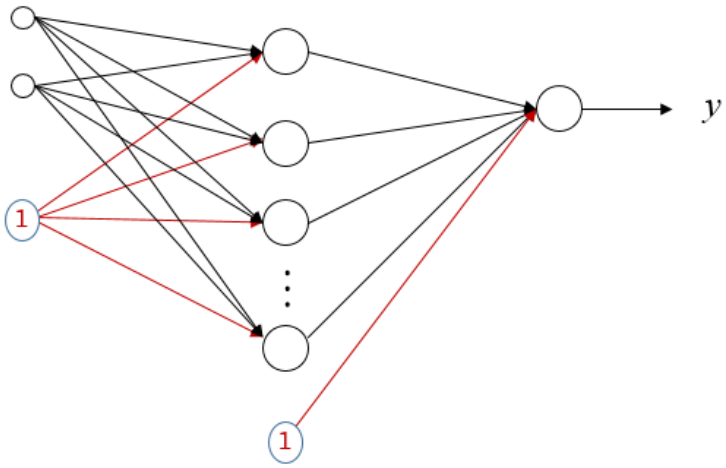
    loss_logs = [] # [size, loss]
    val_loss_logs = [] # [size, val_loss]

    for size in np.arange(from_size, final_size + step_size, step_size):
        (x_data, y_data) = get_number_samples(x_train, y_train, size)

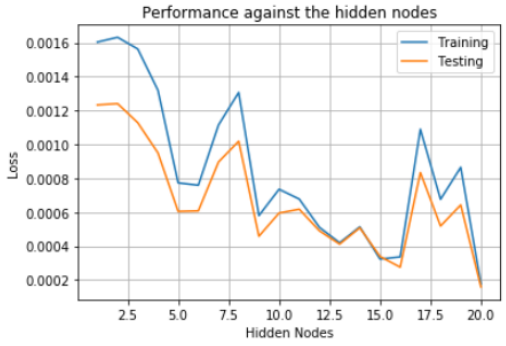
        # Init session
        with tf.Session() as sess:
            sess.run( tf.global_variables_initializer())
            (train_loss, val_loss,  _, _, _, _) = traininig_multilayer_perceptron(x_data, y_data, x_test,
y_test, x_input, y_output, f_loss, train_step, batch_size, epochs, learning_rate)
            loss_logs.append([size, train_loss])
            val_loss_logs.append([size, val_loss])

        csv_log_file.writerow([size, train_loss, val_loss])
        log_file.flush()
    # for
    clear_output(wait = False)
    plot_loss(loss_logs, val_loss_logs, xlabel = 'Size', ylabel = 'Loss', title='Performance against the #
training samples', save_image = save_image)
    log_file.close()
# def
```

3.2. Experiment 2: Performance against the # hidden nodes



- The original dataset (5000 samples)
- MLP with 1 hidden layer with B hidden nodes from 1, 2, …, 20
- Training 100 epochs, 32 batch size




```
size,loss,val_loss
1,0.00160442390442,0.00123322986066
2,0.00163249797821,0.00124079026282
3,0.00156329231262,0.00112805791199
4,0.00131939163208,0.000949661061168
5,0.000772923374176,0.000604568421841
6,0.000758804559708,0.0006075283885
7,0.00111467266083,0.000896135345101
8,0.00130687942505,0.00101891070604
9,0.000579434585571,0.000457697436213
10,0.000735116577148,0.000594691857696
11,0.000677264881134,0.000617421865463
12,0.000510149240494,0.000489637255669
```

With increasing hidden nodes in MLP, loss decreases.

Support Functions

```
# Parameters of MLP
n_input   = 2
n_classes = 1
```

```
n_layers  = 1
n_hiddens = [1, 1]

# Parameters of Training
learning_rate = 0.01
batch_size = 32
epochs = 100

step_size = 1
from_size = 1
final_size = 20

def experiment2(save_image='experiments2.png', log_file_name = 'experiment2.csv'):
    # sess.close()
    global n_input, n_classes, n_layers, n_hiddens
    global learning_rate, batch_size, epochs
    global step_size, from_size, final_size

    # Open log files
    log_file = open(log_file_name, 'wt')
    csv_log_file = csv.writer(log_file, delimiter=',', quotechar='|', quoting=csv.QUOTE_MINIMAL,
lineterminator='\n')
    csv_log_file.writerow(['size','loss','val_loss'])
    log_file.flush()

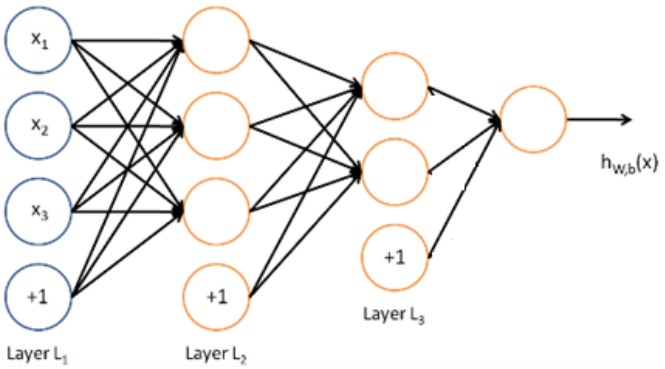
    loss_logs = [] # [size, loss]
    val_loss_logs = [] # [size, val_loss]

    for size in np.arange(from_size, final_size + step_size, step_size):
        n_hiddens[0] = n_hiddens[0] + 1
        n_hiddens[1] = n_hiddens[1] + 1
        (x_input, y_output, mlp_net, f_loss, train_step) = init_multilayer_perceptron(n_input, n_classes,
n_layers, n_hiddens)

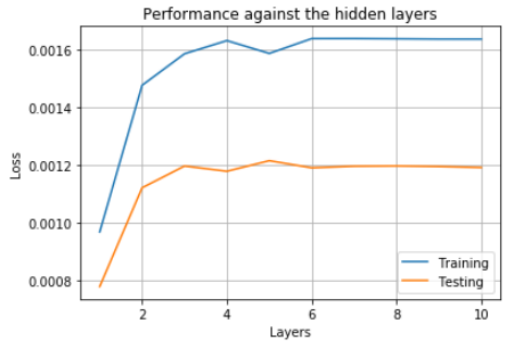
        # Init session
        with tf.Session() as sess:
            sess.run( tf.global_variables_initializer())
            (train_loss, val_loss,  _, _, _, _) = traininig_multilayer_perceptron(x_train, y_train, x_test,
y_test, x_input, y_output, f_loss, train_step, batch_size, epochs, learning_rate)
            loss_logs.append([size, train_loss])
            val_loss_logs.append([size, val_loss])

        csv_log_file.writerow([size, train_loss, val_loss])
        log_file.flush()
    # for
    clear_output(wait = False)
    plot_loss(loss_logs, val_loss_logs, xlabel = 'Hidden Nodes', ylabel = 'Loss', title='Performance against the
hidden nodes', save_image = save_image)
    log_file.close()
# def
```

3.3. Experiment 3: Performance against the # hidden layers



- The original dataset (5000 samples)
- MLP with C hidden layer with 10 hidden nodes from 1, 2, …, 10
- Training 100 epochs, 32 batch size



experiment3.csv

```
size,loss,val_loss
1,0.000967244148254,0.000777662619948
2,0.00147659263611,0.00112063527107
3,0.00158596534729,0.00119614027441
4,0.00163176612854,0.00117801688612
5,0.00158723640442,0.0012147539854
6,0.00163873519897,0.00118991687894
7,0.00163881206512,0.0011955974251
8,0.00163804779053,0.00119666069746
9,0.00163707084656,0.00119452342391
```

Increase layer, loss increases
Vanishing gradient problem

Support Functions

```
# Parameters of MLP
n_input   = 2
n_classes = 1
n_layers  = 1
n_hiddens = [10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10, 10]

# Parameters of Training
learning_rate = 0.01
batch_size = 32
epochs = 100

step_size = 1
from_size = 1
final_size = 10

def experiment3(save_image='experiments3.png', log_file_name = 'experiment3.csv'):
    # sess.close()
    global n_input, n_classes, n_layers, n_hiddens
    global learning_rate, batch_size, epochs
    global step_size, from_size, final_size


    # Open log files
    log_file = open(log_file_name, 'wt')
    csv_log_file = csv.writer(log_file, delimiter=',', quotechar='|', quoting=csv.QUOTE_MINIMAL,
lineterminator='\n')
    csv_log_file.writerow(['size','loss','val_loss'])
    log_file.flush()

    loss_logs = [] # [size, loss]
    val_loss_logs = [] # [size, val_loss]

    for size in np.arange(from_size, final_size + step_size, step_size):
        (x_input, y_output, mlp_net, f_loss, train_step) = init_multilayer_perceptron(n_input, n_classes,
n_layers, n_hiddens)

        # Init session
        with tf.Session() as sess:
            sess.run( tf.global_variables_initializer())
            (train_loss, val_loss,  _, _, _, _) = traininig_multilayer_perceptron(x_train, y_train, x_test,
y_test, x_input, y_output, f_loss, train_step, batch_size, epochs, learning_rate)
            loss_logs.append([size, train_loss])
            val_loss_logs.append([size, val_loss])

        csv_log_file.writerow([size, train_loss, val_loss])
        n_layers = n_layers + 1
        log_file.flush()
    # for
    clear_output(wait = False)
    plot_loss(loss_logs, val_loss_logs, xlabel = 'Layers', ylabel = 'Loss', title='Performance against the
hidden layers', save_image = save_image)
    log_file.close()
# def
```

References

[1] Lectures of Topics in Deep Learning, 2018
[2] Nguyen Hai Dương, Introduction to Deep Learning, 2018